CareLoop

SCHEDULING AGENT / DESIGN NOTE / 12 OCT 2026

TRAINING	VALIDATION	REGRESSED CHECKS
28/32 -> 32/32	**10/12 -> 12/12**	**0**

Scope. The clinic has two fictional patients, three doctors, and weekday appointments in IST. Gemini interprets scheduling requests; an offline parser is also available. DOB matching is simulated verification. The application has no real patient or clinic integration.

Agent. A state machine keeps the patient, request, available slots, and exact booking preview across turns. Gemini returns structured intent; Python validates it and controls the tools. Booking requires verified identity, a preview, and a later explicit confirmation. Corrections invalidate the preview. SQLite transactions, unique slots, and idempotent booking requests prevent duplicate writes. Receipts come from committed records. This keeps booking authority outside model-generated text.

Evaluation. The suite has 32 training and 12 separate validation conversations. Cases cover preferences, dates, corrections, ambiguous assent, identity failures, stale slots, outages, and injection attempts. Every check must pass for a scenario to pass. Checks inspect the transcript, tool audit, and database state. Validation is withheld from the repair proposer, but its cases were reviewed during development and now serve as regression checks rather than an independent unseen benchmark.

Improvement loop. Gemini receives actual failed training transcripts and proposes typed policy settings or new interpretation guidance. The candidate is replayed on training and then validated. Promotion requires a higher training score, all safety checks passing, and no previously passing check lost on either split. The offline workflow is also checked for regressions. The loop cannot change executable code, tests, model weights, or booking safeguards. Existing chats keep their original policy; rejected or interrupted runs keep the active policy.

Results. The recorded Flash-Lite experiment improved training from 28/32 to 32/32 and validation from 10/12 to 12/12, with zero regressed checks. Gemini enabled soft-preference ranking and one read retry. The run used one live proposal request and 100 cached interpretation responses, so these scores are not independent repeated samples. Reports and transcripts are in artifacts/gemini/browser-run/.

Human judgment and AI help. Codex helped write the application, scenarios, and tests. I rejected the confirmation flow that required another slot selection after an ambiguous reply and asked for it to be corrected. That manual Python fix preserves an unchanged preview, leaves the appointment unbooked, and still requires a later exact "yes". It is separate from Gemini's generated policy changes. I also chose Gemini for the model integration. API keys stay server-side and are excluded from the submission.

Before production. Replace simulated verification with authenticated clinic identity and use clinician-reviewed urgency escalation. Add a real scheduling integration, monitoring, and independent representative evaluations. The current urgency guard and synthetic scores do not establish clinical readiness.